ĐỒ ÁN TIN HỌC – HỌC KỲ I (2026-2027)

XÂY DỰNG CHƯƠNG TRÌNH QUẢN LÝ THƯ VIỆN

BẰNG NGÔN NGỮ C++

Giảng viên: Lương An Vinh

SVTH: Phạm Đăng Lưu - Trần Đức Minh

Lớp: D24_TH05

1. Kiến trúc chương trình:

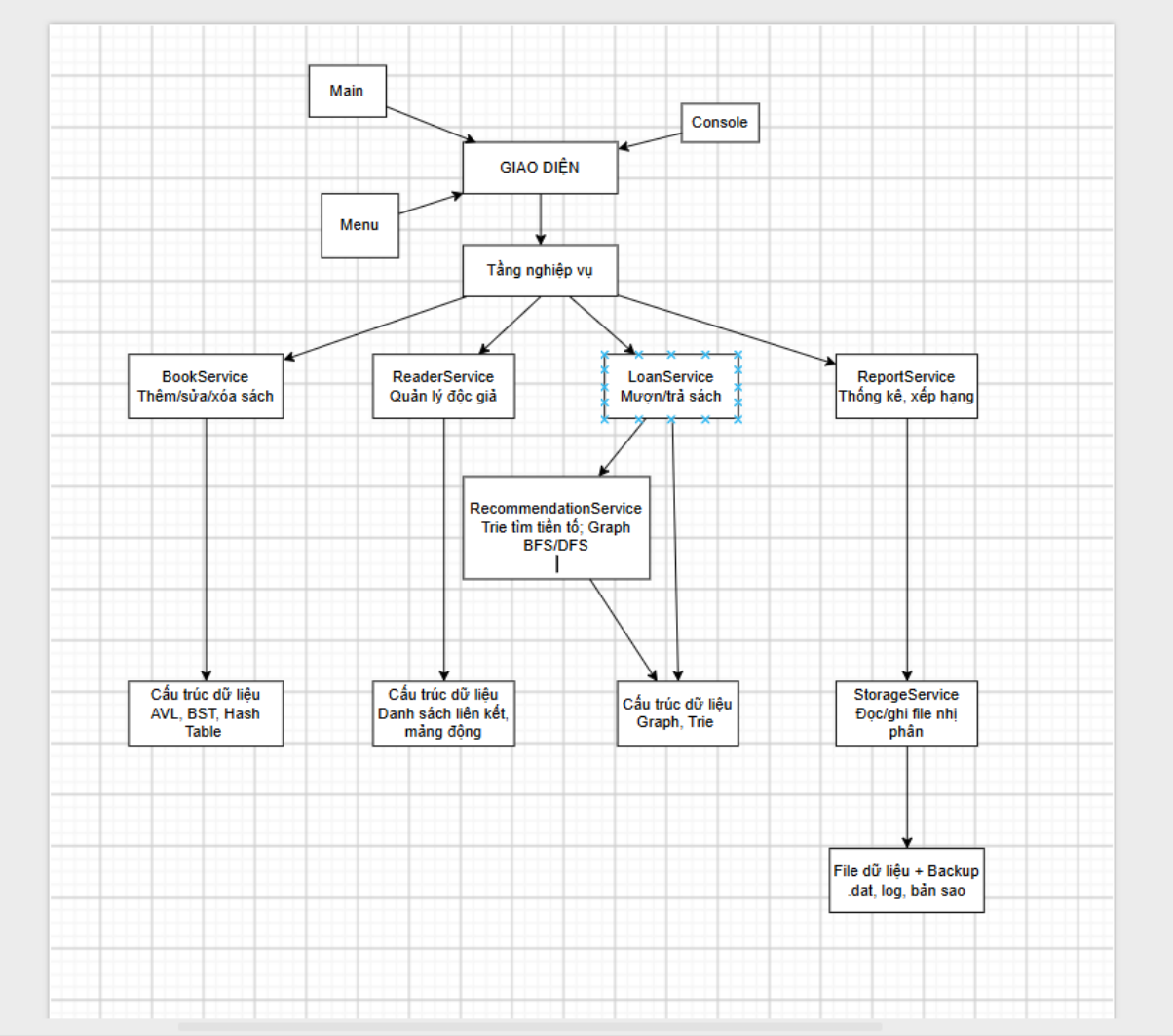
– Chương trình được chia làm 4 tầng: giao diện, nghiệp vụ, cấu trúc dữ liệu, và lưu trữ.

- **Giao diện:** menu, nhập liệu.
- **Nghiệp vụ:** các service kiểm tra ràng buộc.
- **Cấu trúc dữ liệu:** tự cài đặt.
- **Lưu trữ:** đọc file, ghi file, sao lưu.

– Toàn bộ dữ liệu sẽ nằm trong 1 đối tượng Library truyền cho các service và không cần biến toàn cục.

2. Vẽ sơ đồ lớp / sơ đồ module:

a. Sơ đồ module



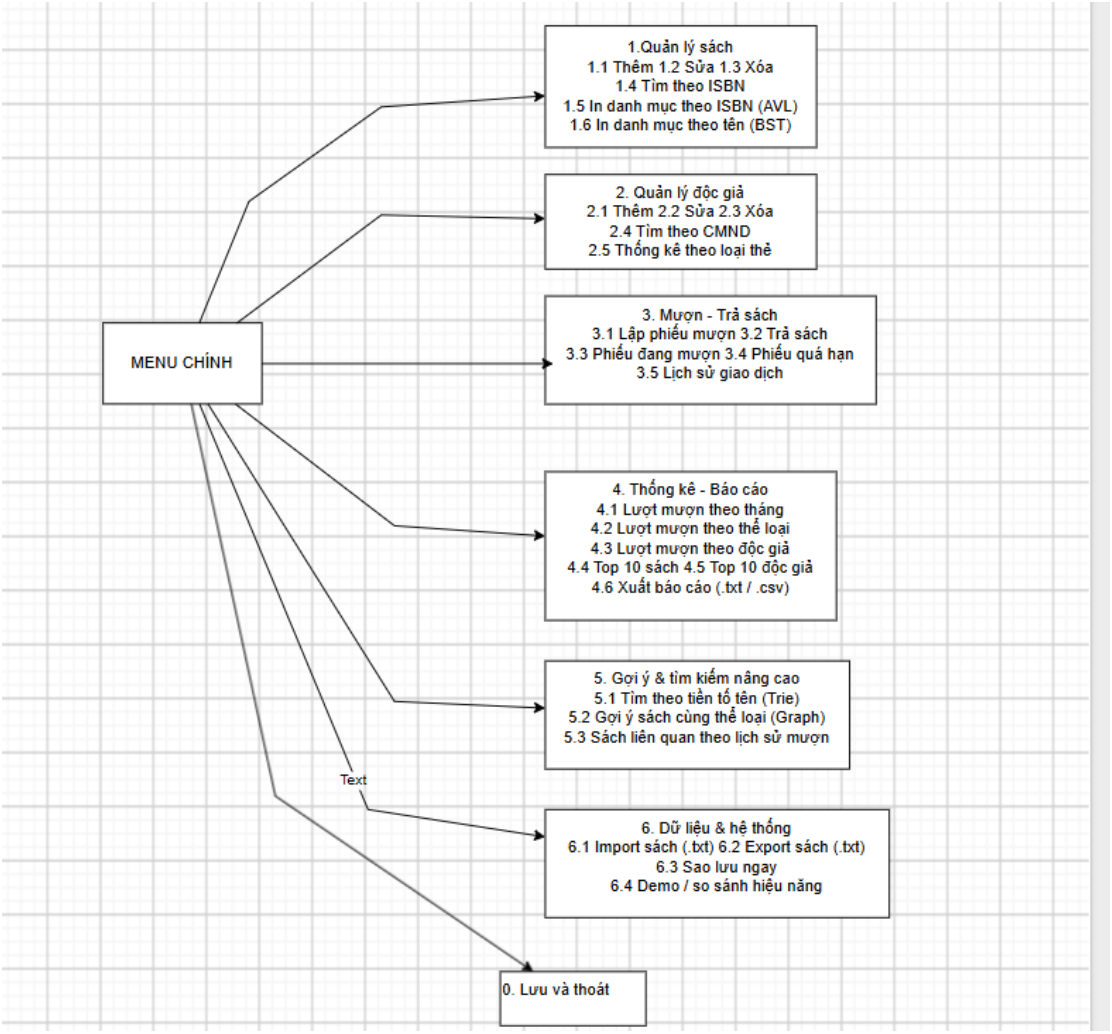
3. Cấu trúc dữ liệu cho từng chức năng:

CTDL	Dùng cho chức năng	Vì sao chọn	Độ phức tạp
Cây AVL	Quản lý sách theo ISBN (thêm, xóa, sửa, tìm, in theo ISBN)	Tự cân bằng nên tìm / thêm / xóa luôn nhanh; duyệt in-order cho danh sách đã sắp	Tìm, thêm, xóa: O(log n). In danh mục: O(n).
Cây BST	In danh mục sách theo tên	Đề yêu cầu; khóa = tên đã bỏ dấu + ISBN; duyệt in-order	TB O(log n), xấu nhất O(n). In: O(n).
Bảng băm (separate chaining)	Quản lý độc giả theo CMND	Tra cứu theo CMND gần như tức thì; va chạm xử lý bằng danh sách liên kết	TB O(1), xấu nhất O(r).
Danh sách liên kết đơn	Danh sách phiếu mượn	Phiếu luôn thêm vào cuối, không cần dịch phần tử	Thêm cuối O(1); tìm theo mã phiếu O(L).
Mảng động	Lịch sử giao dịch (để thống kê)	Chỉ thêm cuối và duyệt tuần tự; truy cập theo chỉ số	Thêm cuối O(1) khấu hao; duyệt O(T).

Đồ thị (danh sách kề), BFS/DFS	Gợi ý sách cùng thể loại	Đồ thị thưa nên dùng danh sách kề; BFS lấy sách gần nhất trước	O(V + E)
Trie (CTDL tự tìm hiểu)	Tìm sách theo tiền tố tên (autocomplete)	Thời gian tìm chỉ phụ thuộc độ dài tiền tố	Thêm / tìm tiền tố: O(m)
File nhị phân	Lưu toàn bộ dữ liệu	Bản ghi kích thước cố định, đọc ghi nhanh	O(n + r + L + T)

4. Thiết kế menu và luồng chương trình:

a. Thiết kế menu:



b. Luồng chương trình:

5. Định nghĩa các struct dữ liệu:

A) Hằng số và kiểu dùng chung

```
const int ISBN_LEN = 14,  TITLE_LEN = 100, AUTHOR_LEN = 60, GENRE_LEN = 30;
const int CMND_LEN = 13,  NAME_LEN  = 60,  EMAIL_LEN  = 50, PHONE_LEN = 12;
const int LOANID_LEN = 12;
const int HASH_SIZE = 101;      // số bucket của bảng băm (số nguyên tố)
const int LOAN_DAYS = 14;       // số ngày được mượn
const int FINE_PER_DAY = 2000;  // tiền phạt mỗi ngày quá hạn (VNĐ)
const int TRIE_ALPHABET = 37;   // a-z, 0-9, khoảng trắng

struct Date { int day, month, year; };
```

B) Dữ liệu cốt lõi (ghi xuống file nhị phân)

```
struct Book {                   // Sách
   char isbn[ISBN_LEN];         // khóa của cây AVL
   char title[TITLE_LEN];     char
author[AUTHOR_LEN];     char
genre[GENRE_LEN];
   int  year;                   // năm xuất bản     int  quantity;              //
tổng số bản     int  available;             // số bản còn trong kho
   int  borrowCount;            // số lượt mượn (phục vụ thống kê)
};

struct Reader {                 // Độc giả     char
cmnd[CMND_LEN];     // khóa của bảng băm
   char     fullName[NAME_LEN];     char
email[EMAIL_LEN];     char
phone[PHONE_LEN];
   CardType cardType;
   int      borrowing;          // số sách đang mượn
};

struct Loan {                   // Phiếu mượn
   char       loanId[LOANID_LEN];     char
cmnd[CMND_LEN];     char
isbn[ISBN_LEN];
   Date       borrowDate, dueDate, returnDate;   // returnDate = 0/0/0 nếu chưa trả
   LoanStatus status;
   long       fine;             // tiền phạt (VNĐ)
};

struct Transaction {            // Một dòng lịch sử giao dịch
   char      loanId[LOANID_LEN];     char
cmnd[CMND_LEN];     char
isbn[ISBN_LEN];
   TransType type;     Date
date;     long      fine;
};
struct FileHeader {
   char magic[4];               // "LIB1" để nhận biết file hợp lệ
   int  version;
   int  bookCount, readerCount, loanCount, transCount;
};
```

```
// Bố cục: FileHeader | Book[bookCount] | Reader[readerCount] //        |
Loan[loanCount] | Transaction[transCount]
// Bản sao lưu: data/backup/library_YYYYMMDD.dat   Nhật ký: data/log.txt
```

C) Nút và container của các cấu trúc dữ liệu (tự cài đặt)

```
// --- Cây AVL: quản lý sách theo ISBN ---
struct AVLNode { Book data; AVLNode *left, *right; int height; };
struct AVLTree { AVLNode* root; int count; };

// --- Cây BST: in danh mục theo tên (khóa = tên bỏ dấu + ISBN) ---
const int KEY_LEN = TITLE_LEN + ISBN_LEN;
struct BSTNode { char key[KEY_LEN]; Book* book; BSTNode *left, *right; };
struct BSTTree { BSTNode* root; int count; };
// book trỏ tới Book trong AVLNode => khi xóa sách phải xóa khỏi BST trước.
// --- Bảng băm, separate chaining: quản lý độc giả theo CMND ---
struct HashNode  { Reader data; HashNode* next; };
struct HashTable { HashNode* buckets[HASH_SIZE]; int count; };

// --- Danh sách liên kết đơn: phiếu mượn (giữ thêm tail để thêm cuối O(1)) ---
struct LoanNode { Loan data; LoanNode* next; }; struct
LoanList { LoanNode *head, *tail; int size; };

// --- Mảng động (template): lịch sử giao dịch ---
template <typename T>
struct DynArray { T* items; int size; int capacity; };  // đầy thì nhân đôi capacity

// --- Đồ thị danh sách kề: gợi ý sách cùng thể loại (BFS/DFS) ---
struct GraphEdge   { int to; GraphEdge* next; };        // to = chỉ số đỉnh kề struct GraphVertex {
char isbn[ISBN_LEN]; char genre[GENRE_LEN]; GraphEdge* head; };
struct Graph       { GraphVertex* vertices; int vertexCount, capacity; };
// Mỗi sách là một đỉnh; cạnh nối hai sách cùng thể loại.
// --- Trie: tìm sách theo tiền tố tên (CTDL tự tìm hiểu) ---
struct TrieNode {
   TrieNode*       children[TRIE_ALPHABET];
   bool            isEnd;      // kết thúc một tên sách
   DynArray<Book*>  books;      // các sách có cùng tên (nếu isEnd)
};
struct Trie { TrieNode* root; };
```

D) Đối tượng Library (chứa toàn bộ dữ liệu, không dùng biến toàn cục)

```
struct Library {
   AVLTree             books;         // sách theo ISBN
   BSTTree             booksByTitle;   // sách theo tên
   HashTable           readers;       // độc giả theo CMND
   LoanList            loans;         // phiếu mượn
   DynArray<Transaction> history;      // lịch sử giao dịch
   Graph               genreGraph;     // gợi ý cùng thể loại     Trie
titleTrie;      // autocomplete tên sách
};
// Mỗi Service nhận Library& qua tham số: BookService::addBook(Library& lib, const Book& b)
```